Artificial Neural Networks
Coursework Assessment

NB6057CEM | BSc Computer Science with AI

Complete two linked individual courseworks: develop and evaluate neural network solutions to a challenging real-world problem, then explain and critically assess the principles and evidence behind your design decisions. The practical coursework carries 60 marks, and the theory coursework carries 40 marks.

Assessment	Required submission	Marks
CW1 Practical	Executable project, results, technical record and demonstration	100
CW2 Theory	Individual analytical report of approximately 2500 words	100

Learning outcomes assessed

Module outcome	Assessment evidence
LO1 Neural network fundamentals and biological inspiration	CW1 T2; CW2 Q1 and Q2
LO2 Compare architectures and learning approaches	CW2 Q2 and Q3; supported by CW1 T2 and T3
LO3 Design and develop models for real applications	CW1 T1 to T4
LO4 Use neural network tools and appraise limitations	CW1 T3 to T6
LO5 Critically evaluate developments	CW2 Q4

CW1 Practical coursework

Design and evaluate neural networks for a real-world problem

Maximum mark: 100 | Individual project | Primary outcomes: LO1, LO3 and LO4

Project scenario

You are an AI engineer asked to investigate whether a neural network can support a clearly defined decision or prediction task. Select a public, documented dataset and a realistic application. Develop competing models, establish a fair comparison and recommend a solution with evidence of both its strengths and its limitations.

Suitable directions include image classification, demand forecasting, sensor-based activity recognition, text classification, or tabular risk prediction. Choose one task only. Your dataset must provide enough independent examples for meaningful development and held-out evaluation. A small teaching dataset used unchanged with a standard tutorial is insufficient unless you add a substantial, justified investigation.

Minimum technical scope

Models. Implement one appropriate non-neural baseline and two meaningfully different neural network architectures for the same prediction task. At least one neural model must contain two or more trainable hidden or feature-processing layers. A change only in width, optimiser or learning rate does not count as a second architecture.

Architecture choices. Examples include MLP and CNN for images; an MLP using a fixed history window and an Elman RNN or other recurrent model for sequences; or MLP and RBF networks for tabular data. Justify suitability. SOM, spiking, hybrid or modular models may be used where they support the task, but are not all compulsory.

Tools and access. Use Python with an established neural network framework or simulator, such as PyTorch or TensorFlow/Keras. A CPU-feasible project is acceptable; neither paid computing nor a GPU is required. Pretrained models are optional: identify their source, training data when known, and frozen and trainable components.

T1 Problem definition and data preparation

8 marks

a Problem and provenance 2 marks. Define the intended user, prediction target, unit of observation and practical value. Identify the dataset source, licence or access conditions, sample size and potential ethical concerns.

b Data audit 2 marks. Inspect missing values, duplicates, imbalance and relevant data quality issues. Provide at least two informative exploratory plots and explain how they affect your design.

c Leakage-safe pipeline 4 marks. Create justified training, validation and test partitions. Use chronological or grouped splitting when observations are dependent. Fit preprocessing on training data only, apply augmentation only during training, and save split indices and preprocessing settings. Explain how target or identity leakage is prevented.

CW1 Implementation and experiments

T2 Implement and verify learning from first principles

12 marks

Using NumPy without automatic differentiation, implement a one-hidden-layer MLP for a manageable subset or reduced representation of your chosen dataset. This foundation model is separate from the two framework models required in T3. Use vectorised operations where appropriate.

a Forward computation 3 marks. Implement trainable weights and biases, a nonlinear hidden activation, an appropriate output activation and a matching scalar loss. State array shapes and explain how the biological neuron analogy relates to the computational unit.

b Backpropagation 4 marks. Implement analytical gradients and mini-batch gradient descent updates. Support binary, multiclass or regression outputs as appropriate to your task. Report initial and final loss and show the loss curve.

c Gradient verification 3 marks. Compare analytical and central finite-difference gradients for at least five entries spanning both weight matrices and both bias vectors. State the perturbation size, use a small batch, avoid ReLU kink points, and report absolute and relative discrepancies with a justified tolerance.

d Numerical diagnosis 2 marks. Provide evidence from one controlled check, such as overfitting a tiny batch, and interpret the result. Report the learning rate, stopping rule and actual epochs completed. Diagnose failure to converge rather than hiding it.

T3 Develop the baseline and framework models

14 marks

a Baseline 2 marks. Implement a credible non-neural comparator, such as logistic regression, a tree-based model or a forecasting baseline. Train it on the same permitted data and evaluate it using the same primary metric.

b Two neural architectures 6 marks. Implement and train two distinct architectures, 3 marks each. Supply architecture diagrams or layer tables showing input and output shapes, activations and trainable parameter counts. Explain depth, width, output layer, loss, initialisation and optimiser choices.

c Training control 3 marks. Implement and document batching, learning rate, regularisation and a validation-based checkpoint or early-stopping procedure. Save the selected weights and provide training and validation loss curves for both architectures.

d Traceable implementation 3 marks. Separate preparation, training and evaluation logic; identify borrowed code and pretrained components. Record configuration, framework version and compute environment. Show that each reported model corresponds to a saved configuration and checkpoint.

Fair comparison rule

Use identical data partitions and comparable preprocessing information across models. Define a modest tuning budget before searching, disclose trials and choose models using validation evidence only. Do not alter the project after inspecting test outcomes. Explain unequal compute budgets or model-specific preprocessing where unavoidable.

CW1 Evaluation and submission

T4 Conduct controlled experiments and evaluate results

14 marks

a Repeated evaluation 4 marks. Run each final neural configuration with three stated random seeds. Keep data splits fixed; vary initialisation and training randomness. Report individual results and mean with standard deviation. Freeze configurations before test evaluation and do not select the best seed using test results. Report the baseline alongside them.

b Metrics and plots 4 marks. Justify a primary metric and at least one complementary metric. For classification, include a confusion matrix and class-level precision, recall and F1; add ROC or precision-recall analysis when appropriate. For regression or forecasting, include MAE or RMSE, predicted-versus-observed and residual plots; preserve time order for forecasting. Label all axes and explain the patterns.

c Ablation 3 marks. Test one design claim by changing a single factor in the selected architecture, for example removing dropout or varying depth while retaining other settings. Compare on validation data using the same three seeds. Report the effect and identify remaining confounders.

d Error and robustness analysis 3 marks. Inspect at least five representative errors and perform one justified stress test, such as input noise or a meaningful subgroup comparison. State the protocol in advance, report sample sizes and explain what the evidence can and cannot establish.

T5 Recommend a model and demonstrate its use

8 marks

a Model recommendation 3 marks. Balance predictive quality, variability, parameter count and measured inference time under a stated batch size and hardware setting. Explain why your recommendation follows from the evidence, including cases where the simpler model is preferable.

b Inference pipeline 3 marks. Provide a command-line or notebook interface that loads the saved model and preprocessing, accepts an unseen example and returns an interpretable prediction. Demonstrate one normal input and one invalid or out-of-scope input with suitable handling.

c Limitations 2 marks. Explain dataset bias, generalisation limits, privacy or misuse concerns relevant to the application, and one feasible next experiment. Do not claim deployment readiness from a single benchmark.

T6 Reproducibility and technical demonstration

4 marks

a Reproduction package 2 marks. Provide a README with setup and run commands, pinned dependencies, data acquisition instructions, saved splits, seeds, configurations, checkpoints and machine-readable metrics. Include a short smoke-test command; do not redistribute restricted data.

b Demonstration 2 marks. Give a 5 to 7 minute demonstration, live or recorded as specified by the lecturer, showing inference and tracing one result from configuration to output. Explain one design choice and one observed failure. Assessment concerns technical understanding, not video production.

CW2 Theory coursework

Neural network principles and critical analysis

Maximum mark: 100 | Individual report | Primary outcomes: LO2 and LO5; supporting LO1

Write an analytical report of approximately 2500 words, excluding references, equations, tables and appendices. Answer Q1 to Q4 and relate your arguments to CW1 where requested. Use labelled diagrams, mathematical notation and scholarly evidence. Explain why choices work and where they fail; do not submit a descriptive catalogue of architectures or repeat the practical technical record.

Q1 Fundamentals and numerical backpropagation

10 marks | Suggested allocation: 450 words plus calculations

a Foundations 3 marks. Explain biological inspiration and two limitations of the analogy. Distinguish a single-layer perceptron from an MLP, explain why a linear decision boundary cannot solve XOR, and show how nonlinear hidden units change representational capacity.

For parts b and c, use one input $x = [1, 2]^T$ and target $y = 1$. The network is $z = Wx + b$; $h = \mathrm{ReLU}(z)$; $\hat{y} = v^T h + c$. The output is linear and the loss is $L = \frac{1}{2}(\hat{y} - y)^2$. The hidden rows of W are [0.2, −0.1] and [0.1, 0.3]; $b = [0.1, -0.2]^T$; $v = [0.4, -0.5]^T$; $c = 0.2$. Use learning rate $\eta = 0.1$.

b Forward pass 2 marks. Calculate both hidden pre-activations, both hidden activations, the prediction and the scalar loss. Show intermediate values and distinguish prediction error from loss.

c Backward pass and update 4 marks. Derive and calculate gradients for every element of W, b, v and c using the chain rule. Perform one simultaneous gradient-descent update using gradients at the original parameters. Recompute the prediction and loss. Explain the three factors in a hidden-unit error signal and why backward gradient computation does not make this a recurrent network.

d Interpretation 1 mark. Explain why one loss-reducing step does not establish convergence or generalisation, and identify one relevant stopping criterion.

Q2 Compare architectures and learning approaches

12 marks | Suggested allocation: 750 words

a Architecture comparison 6 marks. Compare MLP, RBF, Kohonen SOM, Elman recurrent and spiking neural networks in a concise matrix. Address information representation, connectivity or state, learning mechanism, supervision, a suitable application and a limitation. Distinguish temporal state from backpropagation and avoid implying that all spiking networks use one learning rule.

b Learning paradigms 3 marks. Contrast supervised learning and unsupervised competitive learning using MLP backpropagation and SOM as examples. Explain the objectives, label requirements and appropriate evaluation. Explain why a SOM cluster identifier is not automatically a class prediction.

c Applied judgement 3 marks. Recommend and defend architectures for static tabular prediction and online sequential sensor analysis. Explain the role of temporal information, discuss an unsuitable alternative for each, and relate one recommendation to your CW1 problem.

CW2 Critical analysis and evidence

Q3 Justify the practical design and interpret evidence

10 marks | Suggested allocation: 700 words

a Design argument 4 marks. Using your two CW1 neural models, justify the loss, output activation, hidden activations, depth, width, initialisation, optimiser and regularisation. Explain relevant mechanisms such as gradient flow and model capacity. Support the argument with your learning curves, ablation or error analysis rather than generic definitions.

b Experimental validity 3 marks. Critically appraise splitting, leakage control, tuning fairness, repeated runs and metric selection. Distinguish validation-based selection from final testing. Identify at least two threats to the validity of your model comparison and explain their possible effects.

c Framework limitations 3 marks. Evaluate what your chosen neural network framework or simulator makes easier and what it can conceal. Discuss automatic differentiation, reproducibility or nondeterminism, and at least one computational or deployment constraint using project-specific evidence.

Q4 Critically evaluate neural network developments

8 marks | Suggested allocation: 600 words

a Evidence-based comparison 4 marks. Select two developments beyond a conventional MLP. At least one must concern hybrid or modular networks, such as a CNN-RNN system, mixture of experts or neuro-symbolic approach. The other may concern residual or densely connected networks, attention-based models, self-supervised learning or neuromorphic learning. Compare their contribution, empirical evidence, limitations and relevance to your project. Use at least four peer-reviewed sources, including two published within the five years preceding submission.

b Proposed extension 3 marks. Propose one technically plausible extension to CW1 informed by the literature. Specify how components exchange representations or outputs, how the system would be trained, and a falsifiable hypothesis. Define a fair comparator, metric and experiment capable of showing that the extension does not help.

c Critical synthesis 1 mark. Explain one trade-off involving efficiency, interpretability, data requirements or responsible use. Distinguish published evidence from your own proposed benefits; claims that a newer architecture is automatically better receive no credit.

Report requirements

Organise the report under Q1 to Q4. Include a title page with student ID, project title and word count. Use a consistent academic referencing style and cite datasets, software, reused diagrams and external claims. The report should make its main arguments without requiring the assessor to read source code. Cross-reference numbered practical figures or results where useful.

Submission and marking guidance

Submission checklist

Coursework	Submit
CW1 Practical	One ZIP containing source code or notebooks, README, dependency file, configurations, split indices, saved model and preprocessing, metrics, figures, and the demonstration file or permitted link. Include a technical record of no more than 1500 words, excluding tables and captions.
CW2 Theory	One PDF report of approximately 2500 words with Q1 to Q4, calculations, labelled figures, in-text citations and reference list.

In the CW1 technical record, organise evidence under T1 to T6. Include a results table linking each model to its configuration, seed, validation score and test score. Attach the exploration, learning, prediction-quality and diagnostic plots requested in the tasks. Retain unsuccessful experiments that materially affect your conclusions.

Academic integrity and evidence

Submit individual work. Acknowledge all reused code, pretrained weights, data and external assistance. Any use of generative tools must follow the institution’s assessment policy and be declared. You remain responsible for explaining the submitted code, calculations and claims. Do not invent results, citations or experiment logs.